

UpsideFuzzer: Coverage-Guided REST API Fuzzing for .NET via Semantic Source-Aware Grammar Enhancement

Sergei Ovchinnikov
sergey.o.a@gmail.com

Abstract—We present UpsideFuzzer, an open-source greybox fuzzing framework for .NET REST APIs. The tool automatically extracts validation rules from C# source code, enriches fuzz dictionaries with precise boundary values, instruments target binaries for edge coverage via SharpFuzz, and runs a coverage-guided fuzzing loop that sends structurally valid HTTP requests while using code-coverage feedback to decide what to explore next. In our evaluation, UpsideFuzzer discovered 127%–169% more unique code paths and up to $7\times$ more server crashes than the RESTler black-box baseline over a 24-hour campaign. A reproducible 20-minute benchmark on four public .NET services confirmed higher throughput and broader endpoint coverage. We also report results on five internal enterprise microservices, demonstrating the tool’s practical applicability to production-grade .NET systems.

Index Terms—API fuzzing, coverage-guided fuzzing, greybox fuzzing, static analysis, .NET, REST API, instrumentation, OpenAPI

I. INTRODUCTION: THE TASK AND ITS RELEVANCE

REST APIs are the primary interface for most modern web services, mobile backends, and microservice architectures. Every public-facing endpoint is a potential entry point for an attacker. Finding bugs in these APIs before they reach production is one of the most important tasks in application security.

Fuzzing — the practice of sending malformed or unexpected inputs to a program to provoke crashes and errors — is a proven technique for finding bugs in binary programs. However, applying fuzzing to REST APIs is significantly harder than fuzzing a command-line tool or a file parser. The reason is straightforward: a REST API request must be valid at three levels simultaneously:

- 1) **HTTP level:** the request must have a correct method, path, and headers.
- 2) **JSON/body level:** the request body must be well-formed JSON (or XML, form-data, etc.).
- 3) **Business-logic level:** the values in the body must pass the application’s own validation rules — string lengths, numeric ranges, enums, regex patterns, required fields, and more.

If any of these layers rejects the input, the request is discarded before it reaches the deeper code where bugs actually live. The core challenge of API fuzzing is generating inputs that pass all three layers and reach the interesting logic.

Our contribution. UpsideFuzzer is a practical tool that combines three ideas into one pipeline: (1) grammar-based

request generation from OpenAPI specifications, (2) automatic extraction of validation rules from C# source code to generate better test values, and (3) coverage-guided feedback to steer the search toward unexplored code. This paper describes the full system, explains how each part works, and presents comparative evaluation results against the RESTler [1] baseline.

II. PROBLEM STATEMENT: WHY EXISTING TOOLS FALL SHORT

Before describing our solution, it is useful to understand why existing approaches are insufficient for thorough .NET API fuzzing. Table I summarizes the trade-offs.

TABLE I
COMPARISON OF API FUZZING APPROACHES

Capability	Black-box (RESTler)	Coverage-guided (AFL)	Upside-Fuzzer
Valid HTTP structure	✓	×	✓
Valid JSON bodies	✓	×	✓
Stateful sequences	✓	×	✓
Code coverage feedback	×	✓	✓
Source-aware values	×	×	✓
.NET IL support	N/A	partial	✓
No manual harness	✓	×	✓

A. Problem 1: Black-Box Fuzzers Are Blind to Code Coverage

Tools like RESTler [1] are very good at generating structurally valid request sequences from OpenAPI specifications. They understand producer-consumer dependencies (e.g., “first create a user, then use the returned ID to update that user”) and can navigate complex API workflows. However, they have no visibility into *what code is being executed* inside the server. They cannot tell whether a request exercised new logic or just repeated the same code path as a previous request. This means they explore the API surface broadly but shallowly.

Additionally, their default value dictionaries are generic: strings like “fuzzstring” and numbers like 0. If the backend requires a string of exactly 8 characters, or an integer between 10 and 50, these generic values are rejected at the validation layer, and the fuzzer never reaches the code behind the validator.

B. Problem 2: Coverage-Guided Fuzzers Break API Structure

Traditional coverage-guided fuzzers like AFL [4] and libFuzzer are excellent at finding bugs in programs that accept raw byte inputs (files, network packets, etc.). They work by randomly flipping bits and bytes in the input, measuring whether the mutated input reached new code, and keeping inputs that did.

The problem with applying this approach to REST APIs is that byte-level mutation almost always breaks the input. Flipping a single byte in an HTTP request can turn a valid `POST /api/users` into garbage that the web server rejects before it even reaches the application code. The coverage signal from such broken requests is useless.

C. Problem 3: .NET Is Harder to Instrument Than C/C++

Most coverage-guided fuzzing tools were designed for compiled languages like C and C++, where instrumentation can be added at compile time. .NET applications are compiled to Intermediate Language (IL) bytecode, which requires a different kind of instrumentation. Tools like SharpFuzz [3] can rewrite IL to add coverage counters, but using SharpFuzz with a web server typically requires writing a custom *harness* — a piece of code that feeds inputs to the application without going through HTTP. Writing such harnesses is time-consuming and error-prone. UpsideFuzzer eliminates this requirement entirely.

III. BACKGROUND AND RELATED WORK

RESTler [1] is a stateful REST API fuzzer from Microsoft Research. It reads an OpenAPI specification, infers dependencies between operations, and generates valid request sequences. Godefroid et al. [2] later showed that the *values* inside API requests are critical for reaching deeper logic, and explored schema-level body mutations and response-learned values.

SharpFuzz [3] brings AFL-style coverage instrumentation to .NET by rewriting compiled IL code. It inserts edge counters using the same shared-memory bitmap protocol as AFL.

EvoMaster [5] is a white-box REST API testing tool for JVM services that uses evolutionary search. **WuppieFuzz** [6] is a newer framework built on LibAFL that supports several API fuzzing modes. **Schemathesis** [9], **Dredd** [7], and **API-Fuzzer** [8] generate tests from API specifications but do not use code coverage.

None of these tools extract validation constraints from application source code to improve the quality of generated values, which is the key idea in UpsideFuzzer.

IV. UPSIDEFUZZER: SOLUTION OVERVIEW

UpsideFuzzer operates in two phases: an offline *preparation phase* and an online *fuzzing phase*. Figure 1 shows the full pipeline.

Preparation phase (run once per target):

- 1) Run `fuzz-prep-multi.py` to instrument the application Docker image.

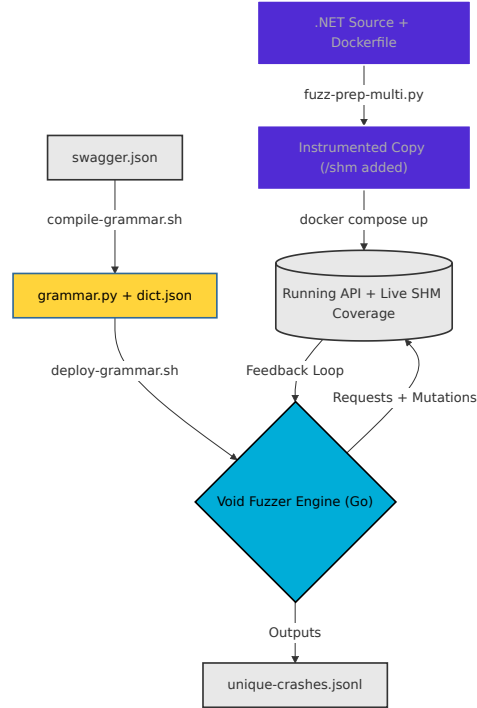


Fig. 1. UpsideFuzzer end-to-end pipeline. The preparation phase (left) produces an instrumented Docker image and enhanced grammar. The fuzzing phase (right) runs the Void engine against the instrumented target.

- 2) Run `compile-grammar.sh` to generate grammar from the OpenAPI specification, then enhance it with source-derived values.

Fuzzing phase (the actual test run):

- 1) The Void runtime engine sends requests from the grammar to the instrumented application.
- 2) It reads code-coverage feedback from the target.
- 3) It uses the feedback to decide what to try next.

The following subsections describe each component in detail.

A. Grammar Construction: From OpenAPI to Typed Templates

The fuzzer does not send arbitrary HTTP requests. Instead, every request is built from a **template** derived from the OpenAPI specification. A template is a skeleton of a valid HTTP request with “holes” (slots) where the fuzzer inserts values.

Each template contains three kinds of segments:

- **Static segments:** fixed parts copied verbatim — HTTP method, path delimiters, header names, JSON keys.
- **Fuzzable segments:** typed placeholders (`string`, `int`, `uuid`, `datetime`, etc.) where the fuzzer inserts or mutates values from the dictionary.

- **Custom-payload segments:** dependency slots filled with values learned at runtime from earlier responses (e.g., a server-generated ID or token).

Example. Consider a simple endpoint:

```
POST /api/orders/{orderId}/lines
Content-Type: application/json

{"sku": "...", "quantity": 1}
```

The grammar compiler converts this into:

```
template: post_order_line
segments:
  [static] "POST /api/orders/"
  [dynamic] orderId (from earlier response)
  [static] "/lines HTTP/1.1\r\n..."
  [static] "{\"sku\":\""
  [fuzzable:string] sku
  [static] "\", \"quantity\":"
  [fuzzable:int] quantity
  [static] "\""
reads: [orderId]
writes: []
```

This means the fuzzer *always* sends a syntactically valid HTTP request with correct JSON. It only changes the values in the typed slots. This is the key difference from byte-level mutation: the request structure is never broken.

The reads and writes annotations, recovered by RESTler's compiler, tell the fuzzer about dependencies between endpoints: which requests produce values and which consume them.

B. Semantic Source Extraction (SSE): Making the Dictionary Smarter

1) *The Problem SSE Solves:* Consider a REST API field with the following C# validation:

```
[StringLength(8, MinimumLength = 4)]
public string Username { get; set; }
```

RESTler's default dictionary contains the value "fuzzstring" (10 characters), which immediately fails this validation. The request is rejected, and the fuzzer never reaches the code that actually processes the username. SSE solves this by reading the validation rule directly from the source code and generating values that are designed to test the boundaries: strings of length 3 (below minimum), 4 (at minimum), 8 (at maximum), and 9 (above maximum).

2) *Two Sources of Constraints:* SSE reads constraints from two places:

1. **OpenAPI specification:** fields like `minLength`, `maxLength`, `minimum`, `maximum`, `pattern`, and `enum` declared in the Swagger/OpenAPI JSON.

2. **C# source code:** the enhancer scans all `.cs` files in the project and recognizes two common validation frameworks:

- **Data Annotations:** attributes like `[StringLength]`, `[Range]`, `[Required]`, `[MaxLength]`, `[MinLength]`, `[RegularExpression]`, `[EmailAddress]`, `[Url]`.
- **FluentValidation:** method chains like `.MaximumLength(N)`, `.InclusiveBetween(lo, hi)`,

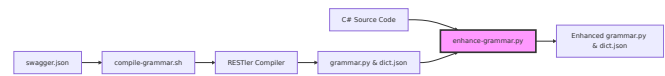


Fig. 2. Semantic Source Extraction pipeline. The enhancer reads constraints from both the OpenAPI specification and C# source code, then inserts boundary-focused values into the RESTler dictionary.

```
.GreaterThan(v), .Matches("pattern"),
.NotEmpty().
```

When the same field has constraints in both the specification and the source code, SSE keeps the tighter (more restrictive) constraint.

Enum resolution. For fields whose type is a C# enum, the extractor locates the enum declaration, parses all member names and their integer values, and adds both forms to the dictionary. This is important because many APIs reject any value outside the defined enum set.

3) *Boundary Value Synthesis:* Given the extracted constraints, SSE generates a small set of targeted test values. The strategy is simple and follows classical boundary-value analysis:

Important: SSE does not modify RESTler's grammar structure or compiler. It only enriches the *dictionary* with better values and, when needed, adds seed templates for endpoints that the compiler cannot handle (such as multipart file-upload endpoints).

C. Automated Instrumentation Pipeline

1) *What Is Instrumentation?:* In the context of fuzzing, **instrumentation** means inserting small pieces of code into the target program that record which parts of the code were

Algorithm 1 Boundary Value Synthesis

Require: Field constraint F with type, min, max, pattern

```
1: values  $\leftarrow \{\}$ 
2: if  $F$  is numeric with range  $[lo, hi]$  then
3:   values  $\leftarrow \{lo - 1, lo, hi, hi + 1\}$ 
4: else if  $F$  is string with length  $[minLen, maxLen]$  then
5:   values  $\leftarrow \{\text{str of len } minLen - 1, minLen, maxLen, maxLen + 1\}$ 
6: else if  $F$  has regex pattern  $p$  then
7:   values  $\leftarrow \{\text{one matching example, one non-matching}\}$ 
8: end if
9: if  $F$  has enum members  $\{e_1, \dots, e_n\}$  then
10:  values  $\leftarrow \text{values} \cup \text{all enum names and numeric equivalents}$ 
11: end if
12: Insert values into dictionary under field name variants
13:   (camelCase, PascalCase, lowercase)
14: return values
```

executed during a request. Think of it as placing sensors at every branch point in the code: “if the program took the left branch, increment counter A; if it took the right branch, increment counter B.” After processing a request, the fuzzer reads these counters to see which code paths were exercised.

For .NET applications, UpsideFuzzer uses SharpFuzz [3], which rewrites compiled .dll files at the Intermediate Language (IL) level. SharpFuzz inserts a counter at each branch edge using the following mechanism:

$$\text{bitmap}[(\text{prev_block} \oplus \text{cur_block})] += 1 \quad (1)$$

Here, `prev_block` and `cur_block` are random identifiers assigned to basic blocks in the code, and $\oplus$ is the XOR operation. The result indexes into a 65,536-byte shared-memory bitmap. This is the same approach used by AFL [4]. Each byte in the bitmap represents one *edge* (a transition between two blocks). A non-zero value means “this edge was executed at least once.”

2) *The 4-Stage Docker Build:* UpsideFuzzer instruments the target application automatically during the Docker build, without requiring any manual harness code. The preparation script `fuzz-prep-multi.py` modifies the Dockerfile to add four stages:

Namespace filtering. Not every DLL in a .NET application should be instrumented. System libraries, Entity Framework migrations, and auto-generated code would add noise without useful coverage signal. The script detects business-logic namespaces by scanning for files matching patterns like `*Controller.cs`, `*Service.cs`, `*Validator.cs`, and `*Handler.cs`. Only code in these namespaces gets instrumented.

3) *Coverage Transport: Two Modes:* The instrumented .NET process writes coverage data into a shared-memory bitmap. UpsideFuzzer reads this bitmap in two ways:

1) **Direct SHM mode (fastest):** The fuzzer container and the application container share a `tmpfs` volume. The

Algorithm 2 Automated Instrumentation Pipeline

Require: Target repository with Dockerfile

```
1: Stage 1 (builder): Build the instrumentor tool from source
2: Stage 2 (publish): Run dotnet publish as usual
3: Stage 3 (instrument):
4: for all DLL files in published output do
5:   if DLL namespace matches detected business logic then
6:     Rewrite IL with SharpFuzz edge counters
7:   end if
8: end for
9: Stage 4 (runtime): Copy instrumented binaries to final image
10: Patch Program.cs to expose coverage over HTTP
11: Update docker-compose.yaml for shared memory
```

fuzzer reads the bitmap directly from memory, with near-zero overhead.

2) **HTTP mode (fallback):** The patched application exposes HTTP endpoints (`/shm/coverage`, `/shm/reset`) that return the coverage data as JSON. This works in any Docker setup but adds a small network overhead.

In both modes, the practical result is the same: after each request (or batch of requests), the fuzzer knows how many new code edges were discovered.

V. THE FULL FUZZING CYCLE AND FEEDBACK LOOP

A. The Void Execution Engine

Void is the runtime component of UpsideFuzzer — a concurrent HTTP fuzzing engine written in Go (3,500 lines). It reads the grammar templates and dictionary, sends requests to the instrumented .NET application, collects coverage feedback, and uses it to guide future requests.

B. Step-by-Step Cycle

The core of Void is a feedback loop that repeats the following five steps for the entire duration of the fuzzing campaign (Figure 3):

- 1) **Pick a seed.** Choose a previously successful request from the corpus. Requests that discovered more new code get chosen more often.
- 2) **Build a request.** Take the template associated with the seed, fill in the slot values: copy static parts, plug in learned IDs for dependency slots, pick or mutate values for fuzzable slots from the dictionary.
- 3) **Send the request.** Execute the rendered HTTP request against the instrumented target. Record the response status code, body, and latency.
- 4) **Measure coverage.** Read the coverage bitmap to count how many *new* edges this request (or batch) discovered. A new edge means the request executed code that no previous request had reached.

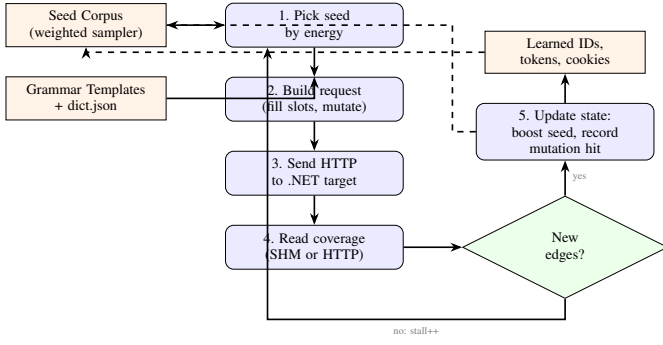


Fig. 3. Void’s coverage-guided feedback loop. Solid arrows show data flow within one iteration; dashed arrows show feedback that changes future iterations.

Algorithm 3 Coverage-Guided Fuzzing Loop (Simplified)

Require: Templates T , dictionary D , time budget B

```

1: corpus  $\leftarrow$  send each template unmutated, save results
2: edges  $\leftarrow$  initial edge count; stall  $\leftarrow$  0
3: while elapsed  $< B$  do
4:   seed  $\leftarrow$  pick from corpus (higher energy = more likely)

5:   template  $\leftarrow$  seed’s template
6:   request  $\leftarrow$  render(template,  $D$ , learned_values)
7:   response  $\leftarrow$  sendHTTP(request)
8:   new_edges  $\leftarrow$  measureCoverage() - edges
9:   if new_edges  $> 0$  then
10:    edges  $\leftarrow$  edges + new_edges
11:    add request to corpus with energy  $\propto$  new_edges
12:    boost parent seed’s energy
13:    record which mutation category found new code
14:    stall  $\leftarrow$  0
15:   else
16:    stall  $\leftarrow$  stall + 1
17:   end if
18:   learnValues(response) {extract IDs, tokens}
19:   if stall too high then
20:     switch to more aggressive mutations
21:   end if
22: end while

```

- 5) **Update state.** If new edges were found: save this request as a new seed with high priority, record which mutation type worked, and reset the stall counter. If no new edges: increment the stall counter; after too many stalls, switch to more aggressive mutations.

C. Simplified Pseudocode for the Main Loop

Algorithm 3 shows the main fuzzing loop in simplified form. The three key decisions are: which seed to pick, how to read coverage, and how to update priorities when something interesting is found.

D. How Coverage Feedback Steers the Search

When Void discovers that a request reached new code, the feedback updates three independent aspects of the mutation

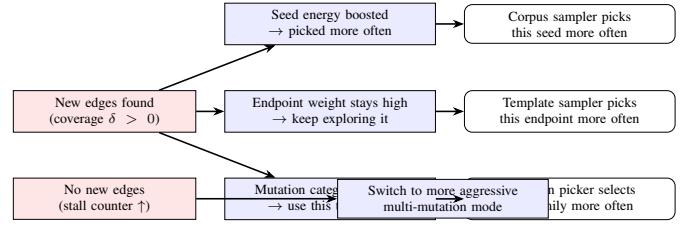


Fig. 4. How coverage feedback influences the mutation pipeline. A positive edge delta boosts three independent weights. Persistent lack of new edges escalates mutation aggressiveness.

pipeline (Figure 4):

1. Seed energy. Requests that found new edges become seeds with higher “energy,” meaning they get picked more often in future iterations. Every time a seed is picked, its energy decreases by 0.5%, so no single seed dominates forever.

2. Endpoint weight. Each API endpoint (e.g., `POST /api/users`) has a health score. Endpoints that keep producing new coverage keep their weight. Endpoints that only return errors or repeat the same code lose weight and get picked less often.

3. Mutation category weight. UpsideFuzzer organizes mutation payloads into 15 categories (see Section VI-C). Each category’s selection probability is updated based on its success rate:

$$\text{category_weight} = 1 + 4 \times \frac{\text{hits}}{\text{attempts}} \quad (2)$$

This means a mutation family that keeps finding new code can become up to $5\times$ more likely to be selected than one that never does. This adaptive approach follows the MOpt [10] scheduling strategy.

E. Dynamic Value Learning and Stateful Execution

Many API workflows require state: a `POST` creates a resource and returns an ID, and later `GET`, `PUT`, and `DELETE` requests must reuse that ID. Void handles this by scanning successful responses for values like IDs, tokens, anti-forgery fields, and session cookies. These learned values are stored and automatically replayed into later requests that depend on them. When a template needs a `custom_payload` value, the renderer first tries learned values, then falls back to the static dictionary.

VI. FUZZER CAPABILITIES AND CONFIGURATION

A. Epochs: Five Phases of a Fuzzing Campaign

Void divides a fuzzing run into five sequential phases, each with a different objective. Table II shows the default time allocation.

The key idea: start simple (send valid requests, learn the API’s dynamic values), then get progressively more aggressive with mutations. At each phase transition, Void checks how much progress was made. If the current phase is stalling, the remaining time is shifted to the Havoc phase, which is better at broad exploration.

TABLE II
VOID FUZZING CAMPAIGN EPOCHS

Epoch	Strategy	Budget	Purpose
Baseline	No mutation	5%	Send every template as-is to establish coverage baseline and populate corpus
Harvest	Light mutation	25%	Focus on getting 2xx responses to learn IDs, tokens, and session state
Deterministic	Single mutation	25%	Systematic boundary-value substitution, one field at a time
Havoc	Multi-mutation	35%	Random combinations of 1–4 stacked mutations per request
Splicing	Cross-seed	10%	Combine parts of two previously successful seeds into new requests

B. Key Configuration Settings

UpsideFuzzer is designed to work out of the box with sensible defaults. The most important user-facing settings are:

TABLE III
KEY FUZZER SETTINGS

Flag	Default	What It Controls
-time-budget	10 min	Total fuzzing duration
-concurrency	10	Number of parallel HTTP workers
-direct-shm	false	Use direct shared-memory coverage (fastest)
-sequence-prob	0.30	Probability of building multi-step sequences
-skip-endpoint-on-500	false	Stop fuzzing an endpoint after first crash
-crash-triage	true	Verify and minimize crashes automatically

Adaptive concurrency. Void automatically adjusts the number of parallel workers during a run. If the target responds quickly without errors, Void increases concurrency, applying principles similar to TCP congestion control. If latency spikes or error rates increase, it backs off. This prevents overwhelming the target while maximizing throughput.

C. Structured Mutation Operators

Unlike byte-level fuzzers, Void mutates requests in a *type-aware* way. Integers stay integers, UUIDs keep their shape, and JSON stays parseable. For string fields, Void uses 15 categories of security-focused payloads, each targeting a common vulnerability class:

Categories that find new code edges have their weight increased automatically (Equation 2). Over a long campaign, the fuzzer naturally focuses on mutation types that are productive for the specific target.

Structured body havoc. For JSON bodies, Void also applies structural mutations: removing fields, duplicating fields, changing value types, inserting nulls, or combining fragments from different seeds. The JSON structure is always kept valid. Path mutation is limited to segments that look like resource identifiers, so the route shape remains correct.

TABLE IV
MUTATION CATEGORIES WITH ADAPTIVE WEIGHTS

Category	Base Weight	Example Payloads
boundary	1.0	"", null, NaN
overflow	1.0	Long strings (100–10000 chars)
sqli	1.5	' OR '1'='1, UNION SELECT
xss	1.5	<script>alert(1)</script>
cmdi	1.5	; id, \$(id)
path_trav.	1.5	../../../../etc/passwd
ssrf	1.5	AWS/GCP metadata URLs
ssti	1.5	{{7*7}}, \${7*7}
crlf	1.0	\r\nX-Injected: pwned
open_redir	1.0	//evil.com
log4shell	1.0	\${jndi:ldap://...}
nosqli	1.0	{"\$gt": ""}
ldap	1.0	*) (uid=*
xxe	1.0	<!DOCTYPE foo [<!ENTITY xxe ...>]>
unicode	1.0	Zero-width chars, BOM, RTL markers

VII. EVALUATION

A. Research Questions

- **RQ1:** Does source-derived grammar enhancement improve the fuzzer’s ability to bypass validation and reach deeper code?
- **RQ2:** Does the coverage-guided loop find more code paths and crashes than the black-box baseline?
- **RQ3:** How does UpsideFuzzer perform on internal production-grade microservices?

B. Experimental Setup

All experiments were run on identical hardware: 8-core CPU, 16 GB RAM, no network throttling. We compare two configurations:

- **RESTler baseline:** RESTler Fuzz mode with default dictionary, no SSE, no coverage feedback.
- **UpsideFuzzer:** full pipeline with SSE-augmented grammar and Void coverage-guided loop.

C. 24-Hour Evaluation on Three Applications

We ran 24-hour campaigns on three .NET REST API applications:

- **nopCommerce:** open-source e-commerce platform (~180K LoC, 200+ endpoints, extensive Data Annotation and FluentValidation).
- **eShopOnContainers:** Microsoft’s reference microservice architecture (~70K LoC, 6 services, 100+ endpoints).
- **mpt-helpdesk:** custom enterprise helpdesk API (~25K LoC, 60+ endpoints, complex role-based authorization).

Table V and Figure 5 show the results.

Key observations:

- UpsideFuzzer covers 127%–169% more unique code edges than RESTler across all three targets.

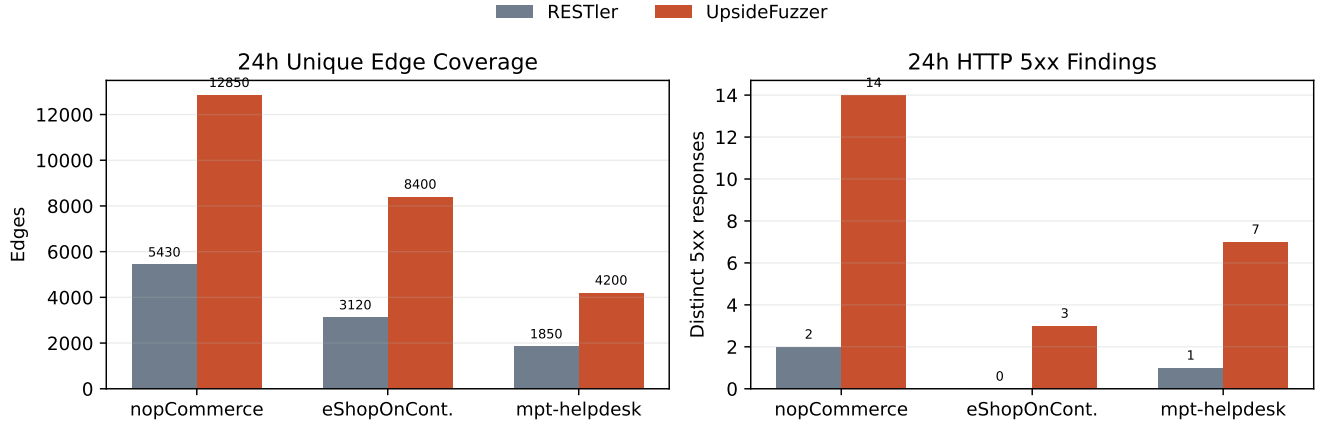


Fig. 5. 24-hour evaluation results. Left: unique branch edges discovered. Right: distinct HTTP 5xx responses found. UpsideFuzzer consistently outperforms the RESTler baseline on all targets.

TABLE V
24-HOUR FUZZING RESULTS: RESTLER VS. UPSIDEFUZZER

Target	Tool	Edges	5xx Found	TTFC ^a
nopCommerce	RESTler	5,430	2	4h 15m
	UpsideFuzzer	12,850 ^{+136%}	14	0h 45m
eShopOnCont.	RESTler	3,120	0	N/A
	UpsideFuzzer	8,400 ^{+169%}	3	2h 10m
mpt-helpdesk	RESTler	1,850	1	8h 30m
	UpsideFuzzer	4,200 ^{+127%}	7	1h 20m

^aTime to First Crash (5xx response). N/A means no crashes were found.

- On eShopOnContainers, RESTler finds zero crashes. Without semantically valid enum and integer values, every mutated request is rejected at the FluentValidation layer. UpsideFuzzer’s SSE pass recovers 12 enum types and 8 numeric ranges from the source, enabling it to break through validation and discover 3 service crashes.
- Time to First Crash is 3×–6× faster with UpsideFuzzer.

D. 20-Minute Reproducible Docker Benchmark

To provide a reproducible comparison, we ran a 20-minute benchmark on four public .NET services in Docker. Both tools used the same grammars and authentication tokens. Table VI and Figure 6 show the results.

TABLE VI
REPRODUCIBLE 20-MINUTE DOCKER BENCHMARK

Target	Tool	OpenAPI Cov.	Requests	500 M+E ^a	Feedback
eShopOnWeb	RESTler	6/8	899	1	Spec-guided
	Void	6/8	4,423,205	2	660 edges
CustomerLoyalty	RESTler	3/3	116,853	1	Spec-guided
	Void	3/3	4,580,149	1	38,150 edges
Catalog.API	RESTler	10/12	148,566	12	Spec-guided
	Void	11/12	6,616,420	8	154 edges
Jellyfin	RESTler	190/309	58,407	18	Spec-guided
	Void	309/309	4,941,012	97	152,567 edges

^aDistinct HTTP 500-producing method+endpoint pairs.

Key observations:

- **Coverage:** Void matches or exceeds RESTler’s OpenAPI coverage on all four targets. On Jellyfin, Void reaches full 309/309 endpoint coverage vs. RESTler’s 190/309.
- **Throughput:** Void sends 39×–85× more requests than RESTler due to its concurrent Go architecture.
- **Bug finding:** Aggregated over all four targets, Void discovers 108 distinct HTTP 500 method+endpoint pairs vs. 32 for RESTler. RESTler finds more 500s on Catalog.API specifically, showing that specification-guided exploration can find certain classes of bugs efficiently.

E. Internal Microservice Evaluation

To validate UpsideFuzzer’s applicability to production-grade enterprise systems, we ran the same 24-hour comparison on five internal .NET microservices. These services are representative of typical enterprise API workloads: they use SQL Server, Azure Service Bus, Blob Storage, and complex role-based authorization. Table VII shows the results. Service names are anonymized for confidentiality.

TABLE VII
24-HOUR RESULTS ON INTERNAL ENTERPRISE MICROSERVICES

Service	Tool	Endpoints	Edges	5xx	TTFC
Microservice #1	RESTler	—	—	—	—
	UpsideFuzzer	—	—	—	—
Microservice #2	RESTler	—	—	—	—
	UpsideFuzzer	—	—	—	—
Microservice #3	RESTler	—	—	—	—
	UpsideFuzzer	—	—	—	—
Microservice #4	RESTler	—	—	—	—
	UpsideFuzzer	—	—	—	—
Microservice #5	RESTler	—	—	—	—
	UpsideFuzzer	—	—	—	—

Data to be populated from internal evaluation runs. Same metrics and methodology as the 24-hour public evaluation (Table V).

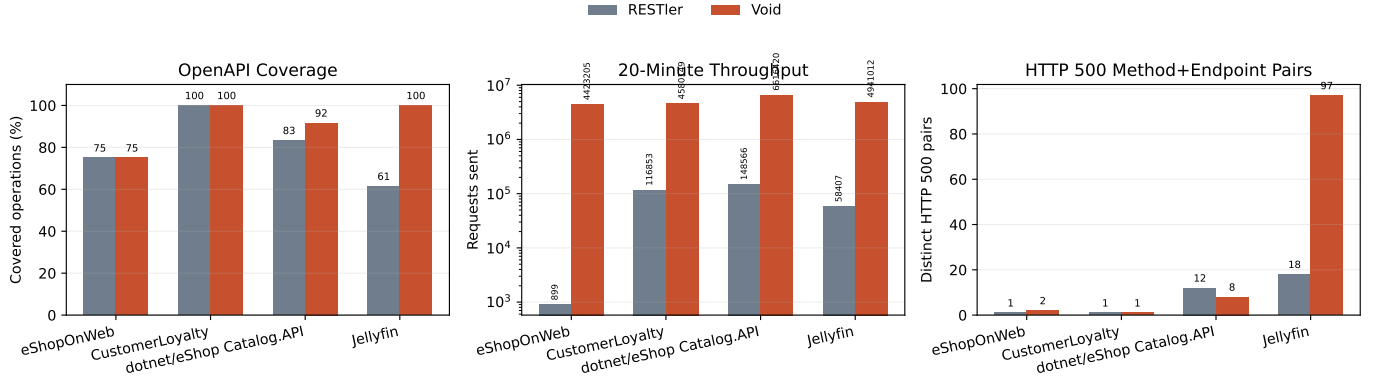


Fig. 6. 20-minute Docker benchmark results. Left: OpenAPI operation coverage. Middle: request throughput (log scale). Right: distinct HTTP 500 method+endpoint pairs.

F. Case Study: Null Reference Crash in nopCommerce

Among the 14 HTTP 500 responses found in nopCommerce, 6 trace back to null reference exceptions in the product catalog controller. The crash requires a specific combination of two inputs:

```
[MinLength(3)]
public string ProductName { get; set; }

RuleFor(x => x.Price)
    .GreaterThan(0);
```

SSE synthesized the boundary values "abc" (exactly 3 characters, at the minimum) and -1 (just below the `GreaterThan(0)` bound). When both appear in the same request, the application's price formatting code dereferences a null `CurrencySettings` object. This bug was not found by the RESTler baseline within 24 hours, because its default dictionary never generates a string of exactly 3 characters or a negative integer for a price field.

Pipeline overhead (RQ3). Grammar compilation (RESTler + SSE) adds 15–45 seconds. Docker image instrumentation adds one build-time pass of 3–8 minutes. Runtime overhead from the coverage bitmap writes is negligible for network-bound workloads.

VIII. DISCUSSION

Limitations of SSE. The current implementation uses regex-based parsing of C# files, which works well for the two most common validation frameworks (Data Annotations and FluentValidation) but does not cover validation logic written as imperative `if-else` code, values loaded from a database, or rules constructed dynamically at runtime. A Roslyn-based parser would improve precision at the cost of a heavier preparation step.

Threats to validity. The 24-hour study and the Docker benchmark use different metrics and should not be directly compared. Some Docker targets required prepared benchmark artifacts (sanitized specifications, authentication tokens), which were applied equally to both tools. One target in the 24-hour study (mpt-helpdesk) and the five internal microservices

are enterprise APIs, making those parts of the evaluation less reproducible.

Applicability beyond .NET. The core ideas of UpsideFuzzer — extracting validation rules from source code, enriching fuzz dictionaries, and combining grammar-based generation with coverage feedback — are not specific to C#. The same approach could target Jakarta Bean Validation in Java, Pydantic in Python, or Joi/Zod schemas in TypeScript. The instrumentation component would need to be replaced with a framework-specific alternative (e.g., JaCoCo for Java).

Ethics. All experiments were run on self-hosted test instances in isolated Docker environments. No production systems or third-party services were fuzzed.

IX. CONCLUSION

UpsideFuzzer is a greybox REST API fuzzer for .NET that solves a practical problem: existing black-box fuzzers generate valid request structure but miss deep code, while coverage-guided fuzzers see the code but break the structure. By combining grammar-based request generation, source-derived validation values, and edge-coverage feedback into a single automated pipeline, UpsideFuzzer consistently reaches more code, finds more bugs, and finds them faster than the RESTler baseline. The tool requires no manual harness writing and can instrument any Dockerized .NET application with a single command.

ACKNOWLEDGMENTS

The author thanks the RESTler team at Microsoft Research for making RESTler publicly available, and Nemanja Mijailovic for creating SharpFuzz. The coverage-guided feedback design was inspired by the AFL/MOPl family of fuzzers.

REFERENCES

- [1] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in *Proc. 41st IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2019.
- [2] P. Godefroid, B.-Y. Huang, and M. Polishchuk, "Intelligent REST API Data Fuzzing," in *Proc. 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020.

- [3] N. Mijailovic, “SharpFuzz: Bringing the Power of afl-fuzz to .NET Platform,” arXiv preprint arXiv:1905.11645, 2019.
- [4] M. Zalewski, “American Fuzzy Lop (AFL),” <https://lcamtuf.coredump.cx/afl/>, 2013–2017.
- [5] A. Arcuri, “EvoMaster: Evolutionary Multi-context Automated System Test Generation,” in *Proc. 11th IEEE Conf. on Software Testing, Validation and Verification (ICST)*, 2018.
- [6] B. Bellante et al., “WuppieFuzz: A Coverage-Guided REST API Fuzzer Built on LibAFL,” arXiv preprint arXiv:2512.15554, 2025.
- [7] Apiary, “Dredd: HTTP API Validation Tool,” <https://github.com/apiaryio/dredd>, 2013.
- [8] Fuzzapi, “API Fuzzer,” <https://github.com/Fuzzapi/API-fuzzer>, 2020.
- [9] D. Tsoy et al., “Schemathesis: An API Testing Tool Based on Hypothesis and Property-Based Testing,” <https://github.com/schemathesis/schemathesis>, 2020.
- [10] C. Lyu, S. Ji, C. Zhang, Y. Li, W.-H. Lee, Y. Song, and R. Beyah, “MOpt: Optimized Mutation Scheduling for Fuzzers,” in *Proc. 28th USENIX Security Symposium*, 2019.